Chapitre 0 — Avant tout | Partie D — Des formes | leçon 0.49 sur 153 | difficulté 1 / 10

0.49. De plus en plus grand : 1, puis 2

Deux carrés autour du même centre, le second un cran plus grand. Le centre (10, 8) est choisi pour que même le plus grand tienne dans l'écran.

1. Ce qu'il faut comprendre

Deux lignes, une seule différence : la taille. D'abord `1`, puis `2`. Le second carré entoure le premier.

Ce qui est nouveau ici : le placement. Pour des carrés de plus en plus grands, il faut un centre où **ils tiennent tous**. L'écran fait **20 colonnes** (de 0 à 19) et **18 lignes** (de 0 à 17). Son milieu est vers la colonne **10** et la ligne **8**.

Pourquoi (10, 8) est le meilleur endroit : le plus grand carré possible a la taille **8** (17 × 17). Autour de (10, 8), il va de la colonne 10 − 8 = **2** à 10 + 8 = **18**, et de la ligne 8 − 8 = **0** à 8 + 8 = **16** : il tient **entier**. Tous les carrés, de la taille 1 à la taille 8, peuvent donc tourner autour du **même** centre, emboîtés comme des cadres.

Chaque carré revient au centre avant que le suivant commence : le second repart donc bien de (10, 8).

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <carre>` → 0.33.3. La fonction carre() — tourner en carré
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <carre>  // une tuile qui tourne en carré`	Demande à la console « carre » (ce que c'est : une tuile qui tourne en carré). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « une tuile qui tourne en carré ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`// LE CENTRE, pour tous les carrés : (10, 8), le milieu de l'écran.`	Un commentaire : une phrase pour celui qui lit (« LE CENTRE, pour tous les carrés : (10, 8), le milieu de l'écran. »). La console l'ignore.
`//  l'écran : 20 colonnes (0 à 19), 18 lignes (0 à 17)`	Un commentaire : une phrase pour celui qui lit (« l'écran : 20 colonnes (0 à 19), 18 lignes (0 à 17) »). La console l'ignore.
`//  le plus grand carré (taille 8) autour de (10, 8) :`	Un commentaire : une phrase pour celui qui lit (« le plus grand carré (taille 8) autour de (10, 8) : »). La console l'ignore.

`//  colonnes 10 - 8 = 2  à  10 + 8 = 18  → il tient (19 est le bord)`	Un commentaire : une phrase pour celui qui lit (« colonnes 10 - 8 = 2 à 10 + 8 = 18 → il tient (19 est le bord) »). La console l'ignore.
`//  lignes  8 - 8 = 0  à  8 + 8 = 16  → il tient (17 est le bord)`	Un commentaire : une phrase pour celui qui lit (« lignes 8 - 8 = 0 à 8 + 8 = 16 → il tient (17 est le bord) »). La console l'ignore.
`//  Donc TOUTES les tailles, de 1 à 8, tiennent autour de ce centre.`	Un commentaire : une phrase pour celui qui lit (« Donc TOUTES les tailles, de 1 à 8, tiennent autour de ce centre. »). La console l'ignore.
`//  Les deux carrés de ce programme, emboîtés :`	Un commentaire : une phrase pour celui qui lit (« Les deux carrés de ce programme, emboîtés : »). La console l'ignore.
`//  colonne :  8  9 10 11 12`	Un commentaire : une phrase pour celui qui lit (« colonne : 8 9 10 11 12 »). La console l'ignore.
`//  ligne 6 :  2  2  2  2  2  1 : le carré de taille 1 (3 × 3)`	Un commentaire : une phrase pour celui qui lit (« ligne 6 : 2 2 2 2 2 1 : le carré de taille 1 (3 × 3) »). La console l'ignore.
`//  ligne 7 :  2  1  1  1  2  2 : le carré de taille 2 (5 × 5)`	Un commentaire : une phrase pour celui qui lit (« ligne 7 : 2 1 1 1 2 2 : le carré de taille 2 (5 × 5) »). La console l'ignore.
`//  ligne 8 :  2  1  A  1  2`	Un commentaire : une phrase pour celui qui lit (« ligne 8 : 2 1 A 1 2 »). La console l'ignore.
`//  ligne 9 :  2  1  1  1  2`	Un commentaire : une phrase pour celui qui lit (« ligne 9 : 2 1 1 1 2 »). La console l'ignore.
`//  ligne 10 : 2  2  2  2  2`	Un commentaire : une phrase pour celui qui lit (« ligne 10 : 2 2 2 2 2 »). La console l'ignore.
`carre(10, 8, ALPHABET[0], 1, 1, 250, 1);  // taille 1 : 3 × 3`	Appel de la fonction carre de la console, avec les arguments 10, 8, ALPHABET[0], 1, 1, 250, 1. Ce qu'elle fait : un carré parfait AUTOUR de (x, y) : taille 1 fait 3 × 3, taille 2 fait 5 × 5 (2 × taille + 1). sens 1 : aiguilles d'une montre, -1 : l'autre sens. vitesse : millisecondes par pas, écrite en clair (250). La lettre part du centre, fait ses tours et y revient. Ou, les sept réglages sous un nom : Carre ronde = { 10, 8, ALPHABET[0], 2, 1, 250, 3 }; carre(ronde);. (Le commentaire le dit : « taille 1 : 3 × 3 ».)
`carre(10, 8, ALPHABET[0], 2, 1, 250, 1);  // taille 2 : 5 × 5, un cran plus loin`	Appel de la fonction carre de la console, avec les arguments 10, 8, ALPHABET[0], 2, 1, 250, 1. Ce qu'elle fait : un carré parfait AUTOUR de (x, y) : taille 1 fait 3 × 3, taille 2 fait 5 × 5 (2 × taille + 1). sens 1 : aiguilles d'une montre, -1 : l'autre sens. vitesse : millisecondes par pas, écrite en clair (250). La lettre part du centre, fait ses tours et y revient. Ou, les sept réglages sous un nom : Carre ronde = { 10, 8, ALPHABET[0], 2, 1, 250, 3 }; carre(ronde);. (Le commentaire le dit : « taille 2 : 5 × 5, un cran plus loin ».)
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();  // le A est revenu au centre`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « le A est revenu au centre ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1380 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  carre()         440 o  ← main()  — une fonction fournie par le langage
  main()           69 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← poser(), effacer()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()   × 4    52 o sur place, dans carre()  + AdresseCase (10 o)
  effacer() × 3    45 o sur place, dans carre()  + AdresseCase (10 o), EffacerCases (12 o)
  image()   × 4    12 o sur place, dans carre(), main()  + AttendreImage (28 o)
Tes fonctions appelées :
  carre()   × 2    62 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <carre>      // une tuile qui tourne en carré
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  // LE CENTRE, pour tous les carrés : (10, 8), le milieu de l'écran.
  //
  //   l'écran : 20 colonnes (0 à 19), 18 lignes (0 à 17)
  //   le plus grand carré (taille 8) autour de (10, 8) :
  //     colonnes 10 - 8 = 2  à  10 + 8 = 18   → il tient (19 est le bord)
  //     lignes    8 - 8 = 0  à   8 + 8 = 16   → il tient (17 est le bord)
  //   Donc TOUTES les tailles, de 1 à 8, tiennent autour de ce centre.
  //
  //   Les deux carrés de ce programme, emboîtés :
  //
  //      colonne :  8  9 10 11 12
  //      ligne 6 :  2  2  2  2  2        1 : le carré de taille 1 (3 × 3)
  //      ligne 7 :  2  1  1  1  2        2 : le carré de taille 2 (5 × 5)
  //      ligne 8 :  2  1  A  1  2
  //      ligne 9 :  2  1  1  1  2
  //      ligne 10 : 2  2  2  2  2

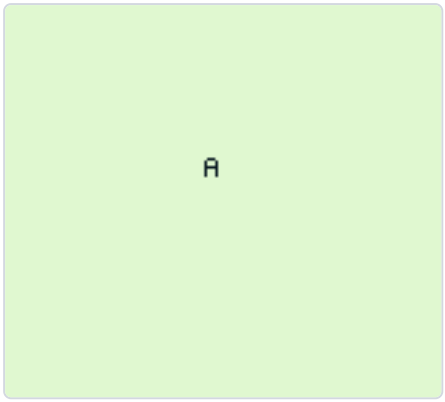
  carre(10, 8, ALPHABET[0], 1, 1, 250, 1);   // taille 1 : 3 × 3
  carre(10, 8, ALPHABET[0], 2, 1, 250, 1);   // taille 2 : 5 × 5, un cran plus loin

  while (true) {
    image();          // le A est revenu au centre
  }
}
```

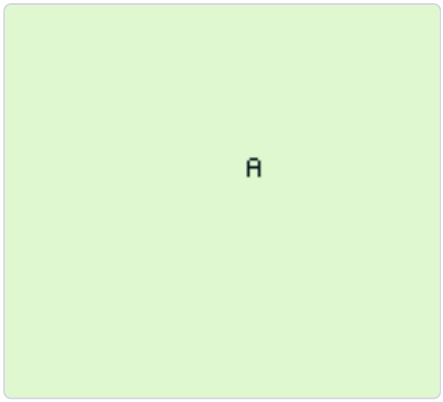
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



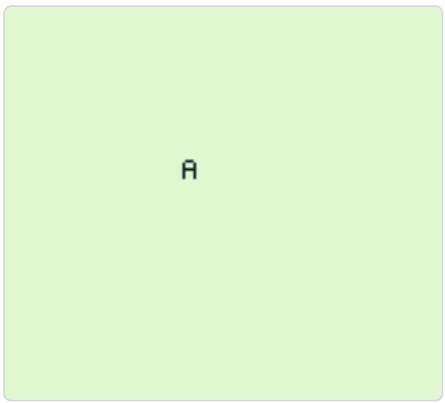
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

> Un A qui fait un petit tour autour du milieu de l'écran, puis un tour un peu plus grand autour du même point.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le carré de taille 1 : son coin (11, 9)
- ✓ puis celui de taille 2 : son coin (12, 10)
- ✓ le A revient au centre (10, 8) (10,8)

Ensuite : **0.49.1. De plus en plus grand : 1, puis 2 — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++